

AKBANK

Java Yazılım Uzmanı

Teknik Mülakat Hazırlık Rehberi

Microservices • Spring Boot • SQL • Java Temelleri

SAGA Pattern • Distributed Systems • Banking Scenarios

Batur için özel hazırlanmıştır

2026

İÇİNDEKİLER

1. Java Temelleri (OOP, SOLID, Exception)
2. Collections & Generics
3. Java 8+ (Stream API, Lambda, Optional)
4. Concurrency & Multithreading
5. JVM, Memory Management & GC
6. Spring & Spring Boot
7. REST API ve Web Service Tasarımı
8. Microservices vs Monolith
9. Microservices Patterns (SAGA, CQRS, Outbox)
10. Inter-Service Communication (Kafka, RabbitMQ)
11. Database & SQL (ACID, Indexes, Transactions)
12. Caching (Redis) ve Performans
13. Security (JWT, OAuth2, Spring Security)
14. Design Patterns
15. Bankacılık Senaryoları ve Soft Skills
16. Son Tüyolar ve Hazırlık Stratejisi

MERHABA BATUR

Bu döküman, Akbank'taki Java Yazılım Uzmanı pozisyonu için yapacağın 1 saatlik teknik mülakata özel olarak hazırlandı. Bankacılık tarafında karşına çıkması en muhtemel sorular, her birinin anlaşılır cevabı ve gerçek hayattan örnekleri (özellikle bankacılık senaryolarıyla) bir arada toplandı.

Nasıl çalışmalısın?

- Her soruyu önce kendi cümlelerinle cevaplamayı dene, sonra burada yazılana bak.
- Cevapları ezberleme; mantığını anla. Mülakatçı seni çevirir, ezbere yakalar.
- Gerçek hayat örneklerini iyice sindir; bunlar seni 'kitap okumuş' biri yerine 'iş yapmış' biri gibi gösterir.
- ARGUS deneyimini de cevaplarına entegre et — leave management, REST service'ler, bulk upload tecrübelerin gerçek senaryolarla örtüşüyor.
- Bilmediğin bir şey çıkarsa, dürüst ol: 'Bunu detaylı kullanmadım ama mantığı şu olmalı...' şeklinde mantık yürüt. Bankalar bunu sever.

Hadi başlayalım, kolay gelsin!

1. JAVA TEMELLERİ

1.1 OOP'nin 4 Temel Prensibi

SORU: OOP nedir? 4 temel prensibini örneklerle anlatır mısın?

OOP (Nesne Yönelimli Programlama), gerçek dünyayı 'nesneler' üzerinden modelleme yaklaşımıdır. Veriyi ve davranışı bir arada tutarız.

1) Encapsulation (Kapsülleme):

Veriyi ve onu kullanan metodları aynı sınıfta toplar, içeri erişimi private, dışa açık metodları public yaparız. Veriyi gizleriz.

□ Gerçek Hayat Örneği: Akbank müşteri hesabını düşün: bakiye alanı private olmalı. Doğrudan hesap.bakiye = -1000 yapılamamalı. Onun yerine hesap.paraCek(500) metodu var ve içeride bakiye yeterli mi, günlük limit aşıldı mı kontrolleri yapılır. ARGUS'ta da izin günü alanına direkt müdahale yerine setter ile validasyondan geçirmek aynı prensiptir.

2) Inheritance (Kalıtım):

Bir sınıf, başka bir sınıfın özelliklerini ve davranışlarını miras alır. Kod tekrarını önler.

□ Gerçek Hayat Örneği: Hesap ana sınıfı: bakiyeGoster(), paraYatir(). VadeliHesap extends Hesap ek olarak faizHesapla(), VadesizHesap extends Hesap ek olarak krediKartiBaglat(). Ortak davranışı yazmazsın, miras alırsın.

3) Polymorphism (Çok Biçimlilik):

Aynı metod farklı sınıflarda farklı davranır. Compile-time (overloading - aynı isim, farklı parametre) ve runtime (overriding - alt sınıf override eder) olmak üzere ikiye ayrılır.

□ Gerçek Hayat Örneği: List<Hesap> hesaplar içine hem VadeliHesap hem VadesizHesap koyarsın. for (Hesap h : hesaplar) h.faizHesapla() dediğinde her biri kendi formülünü çalıştırır. Sen üst tipten konuşursun, JVM doğru implementasyonu seçer. Bu runtime polymorphism.

4) Abstraction (Soyutlama):

Karmaşıklığı gizleyip sadece gerekli arayüzü dışa açmak. abstract class ve interface ile yapılır.

□ Gerçek Hayat Örneği: OdemeServisi interface'i sadece odemeYap(tutar) der. Arkada Visa mı, MasterCard mı, Troy mu kullanıldığını sen bilmek zorunda değilsin.

1.2 SOLID Prensipleri

SORU: SOLID nedir? Açar mısın?

Robert C. Martin'in (Uncle Bob) ortaya koyduğu, sürdürülebilir ve genişletilebilir kod yazmak için 5 prensiptir:

S - Single Responsibility: Bir sınıfın tek bir değişme nedeni olmalı. Yani bir sınıf tek bir iş yapmalı.

□ Gerçek Hayat Örneği: HesapServisi sınıfı içinde hem para transferi, hem PDF dekont üretimi, hem de SMS gönderimi yapma! Bunlar TransferServisi, DekontServisi, BildirimServisi olarak ayrılmalı. Senin ARGUS'taki PdfIzinKarti'nın PDF üretiminden sorumlu olması bu prensibe uyumlu.

O - Open/Closed: Sınıflar genişlemeye açık, değişikliğe kapalı olmalı. Yeni özellik eklerken eski kodu kırıştırmamalısın.

□ Gerçek Hayat Örneği: Yeni bir ödeme yöntemi (Apple Pay) gelecekse, mevcut OdemeServisi'ne if-else eklemek yerine ApplePayStrategy implements OdemeStrategy yazarsın. Strategy Pattern!

L - Liskov Substitution: Bir alt sınıf, üst sınıfın yerine geçtiğinde program doğru çalışmaya devam etmeli.

□ Gerçek Hayat Örneği: Penguen extends Kus ama Kus.uc() metodu varsa, Penguen.uc() exception fırlatırsa Liskov'u ihlal edersin. Tasarımı yeniden düşünmelisin.

I - Interface Segregation: Şişman interface'ler yerine küçük, odaklı interface'ler.

□ Gerçek Hayat Örneği: Tek bir ATMServisi interface'inde para çekme, yatırma, kredi kartı borç ödeme, fatura ödeme hepsi olmasın. NakitIslem, FaturalIslem gibi ayrı interface'ler.

D - Dependency Inversion: Üst seviye modül alt seviye modüle bağımlı olmamalı; her ikisi de soyutlamaya bağlı olmalı. Spring'in Dependency Injection'ının temeli.

□ Gerçek Hayat Örneği: TransferServisi içinde new MailService() yazma! @Autowired BildirimServisi bildirimServisi şeklinde interface'e bağımlı ol. Yarın bildirim için SMS, Push, WhatsApp olsa kodunu değiştirmezsin.

1.3 equals() ve hashCode()

SORU: equals() ve hashCode() arasındaki ilişki nedir? Neden ikisini birlikte override etmeliyiz?

Java'da iki nesnenin 'eşit' olması equals() ile, 'hash kovalasına' yerleştirilmesi hashCode() ile yapılır. Sözleşme (contract):

- İki nesne equals()'a göre eşitse, hashCode()'ları da aynı olmalıdır.
- Hash kodları aynıysa, equals'a göre eşit olmak zorunda değildir (collision olabilir).
- Sadece equals override edersen, HashMap/HashSet çalışmaz.

□ Gerçek Hayat Örneği: Banka müşterisi düşün: Musteri{tckn="12345", ad="Ahmet"}. İki müşteriyi TCKN'ye göre eşit kabul etmek istiyorsun. Eğer sadece equals override edip TCKN'ye bakarsan ama hashCode default kalırsa, HashSet<Musteri> musteriler içinde aynı TCKN'li 2 müşteri olur! Çünkü iki nesne farklı hash kovalasına düşer ve set onları farklı sanır.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Musteri)) return false;
    Musteri m = (Musteri) o;
    return Objects.equals(tckn, m.tckn);
}

@Override
public int hashCode() {
    return Objects.hash(tckn);
}
```

1.4 == ve equals() Farkı

SORU: == ve equals() arasındaki fark nedir?

== referans karşılaştırması yapar (aynı bellek adresini gösteriyorlar mı?). equals() ise içerik (anlamsal) karşılaştırma yapar.

```
String a = new String("Ahmet");
String b = new String("Ahmet");
a == b // false (farklı objeler, farklı adresler)
a.equals(b) // true (içerikleri aynı)

// String literal pool sürprizi:
String x = "Ahmet";
String y = "Ahmet";
x == y // true! (ikisi de pool'daki aynı objeye işaret eder)
```

□ Gerçek Hayat Örneği: Bankada müşteri TCKN karşılaştırırken tckn1 == tckn2 yazma! Veritabanından çekilen String'ler farklı obje olabilir. Her zaman tckn1.equals(tckn2) veya null safety için Objects.equals(tckn1, tckn2).

1.5 String, StringBuilder, StringBuffer

SORU: Bu üçü arasındaki fark nedir?

Özellik	String	StringBuilder	StringBuffer
Mutable?	Hayır (immutable)	Evet	Evet
Thread-safe?	Evet (immutable olduğu için)	Hayır	Evet (synchronized)
Performans	Yavaş (her değişimde yeni obje)	Orta	StringBuilder'dan biraz yavaş
Ne zaman?	Sabit text	Tek thread'de string birleştirme	Çok thread'de string birleştirme

□ Gerçek Hayat Örneği: ARGUS'ta dekont/rapor üretirken yüzlerce satır birleştiriyorsan StringBuilder kullan. String concat (+) ile yaparsan her satırda yeni String objesi oluşur, GC fena yorulur. 10.000 satırlık raporda performans farkı 100x'tir.

1.6 Abstract Class vs Interface

SORU: Abstract class ile interface arasındaki farklar nelerdir? Hangi durumda hangisini kullanırsın?

Özellik	Abstract Class	Interface
Çoklu kalıtım	Tek extends	Çoklu implements
Field	Her türlü field	public static final (sabit)
Constructor	Var	Yok
Metod gövdesi	Abstract ve concrete birlikte	Java 8+ ile default/static var
State (durum)	Tutabilir	Tutamaz
Ne zaman?	IS-A ilişkisi, ortak state var	CAN-DO ilişkisi, davranış kontratı

□ Gerçek Hayat Örneği: Abstract class: abstract class Hesap { String hesapNo; abstract BigDecimal faizHesapla(); void paraYatir(...){...} } - Vadeli/Vadesiz hep aynı 'Hesap'tır, ortak alanları var.

Interface: interface Onaylanabilir { void onayla(); } - Hem IzinTalebi hem KrediTalebi 'onaylanabilir' davranışı sergileyebilir; farklı hiyerarşilerden geliyor olsalar da.

1.7 Checked vs Unchecked Exception

SORU: Checked ve unchecked exception farkı nedir? Hangi durumda hangisi atılır?

Checked Exception: Derleme zamanında (compile-time) kontrol edilir. throws ile bildirmen veya try-catch ile yakalamak zorundasın. Exception'dan miras alır (RuntimeException hariç). Örnek: IOException, SQLException.

Unchecked Exception: Runtime'da fırlatılır, derleyici zorlamaz. RuntimeException'dan miras alır. Örnek: NullPointerException, IllegalArgumentException, ArithmeticException.

□ Gerçek Hayat Örneği: Çoğu modern Spring uygulamasında unchecked tercih edilir. Mesela ARGUS'ta `throw new IzinKuralIhlali("Geriye dönük izin yasaktır")` dersin (RuntimeException'dan türer). Bu sayede metod imzalarını throws bildirimleriyle kirlfetmezsin. Global `@RestControllerAdvice` ile tek noktadan yakalarsın.

```
// Custom unchecked exception
public class YetersizBakiyeException extends RuntimeException {
    public YetersizBakiyeException(String msg) { super(msg); }
}

// Global handler
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(YetersizBakiyeException.class)
    public ResponseEntity<ApiError> handle(YetersizBakiyeException e) {
        return ResponseEntity.status(400).body(new ApiError("INSUFFICIENT_BALANCE",
            e.getMessage()));
    }
}
```

1.8 final, finally, finalize Farkı

SORU: final, finally ve finalize anahtar kelimeleri arasındaki farklar nelerdir?

final: Bir keyword. Değişkende → sabit yapar. Metodda → override edilemez. Sınıfta → extend edilemez (örn. String final'dır).

finally: try-catch bloğunun bir parçası. Exception olsa da olmasa da çalışır. Kaynak temizliği için kullanılır (file close, connection close).

finalize(): Object sınıfının metodu. GC nesneyi temizlemeden önce çağrılır. Java 9'dan itibaren deprecated. Kullanma! try-with-resources veya Cleaner kullan.

1.9 Java Pass-by-Value mi Pass-by-Reference mi?

SORU: Java pass-by-value mi pass-by-reference mi? Açıklar mısın?

Java her zaman pass-by-value'dir. Ama primitive ve object için davranış farklı görünür:

- Primitive: Değerin kopyası gider. Metod içinde değişirse dışarıyı etkilemez.
- Object: Referansın kopyası gider. Yani metod içinde objenin alanlarını değiştirirsen dışarıdan

da görülür (çünkü aynı objeyi gösteriyorsunuz). Ama referansı new ile yeniden atarsanız dışarıdaki referans değişmez.

```
void degisiklik(Musteri m) {  
    m.setAd("Veli"); // Dışarıdan görülür ✓  
    m = new Musteri("X"); // Dışarıdan görülmez ✗  
}
```

1.10 Immutable Nesneler

SORU: Immutable nesne nedir? Nasıl oluşturursun? Avantajları?

Oluşturulduktan sonra state'i değiştirilemeyen nesneye immutable denir. String, Integer, BigDecimal immutable'dır.

Nasıl yapılır?

- 1) Sınıfı final yap (extend ile bozulmasın)
- 2) Tüm field'ları private final yap
- 3) Setter yazma
- 4) Constructor'da defansif kopya al (eğer mutable field varsa)
- 5) Getter'larda defansif kopya döndür

Avantajları: Thread-safe (kilitlemeye gerek yok), HashMap key olarak güvenli, cache'lenebilir, side-effect yok.

□ Gerçek Hayat Örneği: Para alanı için BigDecimal kullanılır (immutable). Eğer mutable olsaydı, para transferinin ortasında bir thread bakiyeyi değiştirebilirdi. Bankacılıkta felaket olurdu. Para hesaplarında asla double/float kullanma, floating-point hatası yüzünden 1 kuruş kayıp bile büyük sorun yaratır.

2. COLLECTIONS & GENERICS

2.1 ArrayList vs LinkedList

SORU: ArrayList ve LinkedList arasındaki fark nedir? Hangisini ne zaman kullanmalısın?

ArrayList: Dinamik dizi. İçinde Object[] var. Belli kapasitede oluşur, dolunca 1.5x büyüyen yeni dizi yapılır ve içerik kopyalanır.

LinkedList: Çift yönlü bağlı liste. Her node bir önceki ve sonraki node'u tutar.

İşlem	ArrayList	LinkedList
get(index)	O(1) - direkt erişim	O(n) - başından gez
add(sona)	O(1) amortized	O(1)
add(ortaya)	O(n) - kaydırma	O(n) - bul + ekle
remove	O(n)	O(1) eğer node'a sahipsen
Bellek	Az (sadece dizi)	Çok (her node 2 pointer)

□ Gerçek Hayat Örneği: %95 durumda ArrayList kullan. LinkedList sadece çok özel durumlarda (örn. Queue/Deque olarak başa-sona sürekli ekleme/silme yapıyorsan) tercih edilir. Pratikte LinkedList neredeyse hiç kullanılmaz.

2.2 HashMap İç Yapısı

SORU: HashMap nasıl çalışır? İç yapısını anlatır mısın?

HashMap arka planda bir dizi (bucket array) kullanır. Default kapasitesi 16'dır.

put(key, value) adımları:

- 1) key.hashCode() alınır.
- 2) Hash değeri üzerinden bucket index'i hesaplanır: (n-1) & hash.
- 3) O bucket'taki LinkedList (Java 8+ 8'den fazla eleman varsa Red-Black Tree) içine eklenir.
- 4) Aynı key varsa equals() ile bulunur ve value güncellenir.
- 5) Load Factor (0.75) aşırsa kapasite 2 katına çıkar ve rehash yapılır (tüm elemanlar yeniden dağıtılır).

Java 8 önemli iyileştirme: Bir bucket'ta 8'den fazla eleman birikirse (çok kötü hashCode ile collision olabilir) LinkedList yerine balanced tree'ye geçer. Bu sayede worst case O(n) yerine O(log n) olur.

```
// Banka müşteri lookup için HashMap
Map<String, Musteri> musteriharitasi = new HashMap<>();
musteriharitasi.put("12345678901", new Musteri("Ahmet"));
// O(1) - direkt bucket'a git, equals ile karşılaştır
Musteri m = musteriharitasi.get("12345678901");
```

2.3 HashMap vs Hashtable vs ConcurrentHashMap

SORU: HashMap, Hashtable ve ConcurrentHashMap farkları nelerdir?

Özellik	HashMap	Hashtable	ConcurrentHashMap
Thread-safe	Hayır	Evet (synchronized)	Evet (segment lock)
Null key/value	İzinli	Yasak	Yasak
Performans	Hızlı	Yavaş (tüm map lock)	Hızlı (parçalı lock)
Eskimişlik	Aktif	Legacy	Modern

□ Gerçek Hayat Örneği: Single-thread uygulamada HashMap. Multi-thread'de ConcurrentHashMap. Hashtable artık kullanılmıyor (legacy). Mesela Akbank'ta bir cache servisi yazıyorsun, binlerce thread aynı anda okuyup yazıyor → ConcurrentHashMap kullan, tüm map'i kilitlemez, sadece etkilenen segment'i kilitler.

2.4 Fail-Fast vs Fail-Safe Iterator

SORU: Fail-fast ve fail-safe iterator nedir?

Fail-Fast: Iteration sırasında collection değiştirilirse anında ConcurrentModificationException fırlatır. Örnek: ArrayList, HashMap iterator'ları. İçerideki modCount değişkenini takip ederler.

Fail-Safe: Collection'ın kopyası üzerinden iterate eder, değişiklikler iterator'ı etkilemez. Örnek: CopyOnWriteArrayList, ConcurrentHashMap.

```
List<String> liste = new ArrayList<>(List.of("a", "b", "c"));
for (String s : liste) {
    if (s.equals("b")) liste.remove(s); // ConcurrentModificationException!
}

// Doğru yol:
Iterator<String> it = liste.iterator();
while (it.hasNext()) {
    if (it.next().equals("b")) it.remove(); // OK
}
```

3. JAVA 8+ ÖZELLİKLERİ

3.1 Stream API

SORU: Stream API nedir? Avantajları ve dezavantajları nedir?

Stream API, koleksiyonlar üzerinde fonksiyonel tarzda işlem yapmamızı sağlar. filter, map, reduce, collect gibi operasyonlarla zincirleme (chain) kod yazarız.

İki tür operasyon var:

- Intermediate (ara): filter, map, sorted, distinct... → Yeni stream döner, lazy'dir.
- Terminal (son): collect, count, forEach, reduce, findFirst... → Stream'i tüketir, sonuç üretir.

```
// Akbank: Son 30 günde 10.000 TL üstü işlem yapan müşterilerin TCKN'leri
List<String> tcknler = islemler.stream()
    .filter(i -> i.getTarih().isAfter(LocalDate.now().minusDays(30)))
    .filter(i -> i.getTutar().compareTo(new BigDecimal("10000")) > 0)
    .map(i -> i.getMusteri().getTckn())
    .distinct()
    .sorted()
    .collect(Collectors.toList());
```

Avantaj: Okunabilirlik, paralel işleme (.parallelStream()), fonksiyonel stil.

Dikkat: Stream bir kez kullanılır. Tekrar iterate edemezsin. Çok küçük listeler için for daha hızlı olabilir.

3.2 Lambda Expressions ve Functional Interface

SORU: Functional interface nedir? Lambda ile ilişkisi?

Functional Interface, tek bir abstract metod içeren interface'tir. @FunctionalInterface annotation'ı ile işaretlenir (zorunlu değil ama güvenli).

Built-in functional interface'ler:

- Function<T, R> → T alır, R döner
- Predicate<T> → T alır, boolean döner
- Consumer<T> → T alır, dönüş yok
- Supplier<T> → Parametre yok, T döner
- BiFunction<T, U, R> → 2 girdi, 1 çıkış

```
Predicate<Hesap> aktifMi = h -> h.getDurum() == AKTIF;
Function<Hesap, BigDecimal> bakiye = Hesap::getBakiye;

BigDecimal toplamAktifBakiye = hesaplar.stream()
    .filter(aktifMi)
    .map(bakiye)
    .reduce(BigDecimal.ZERO, BigDecimal::add);
```

3.3 Optional

SORU: Optional nedir? NullPointerException'dan nasıl korur?

Optional<T>, bir değer olabileceğini veya olmayabileceğini açıkça ifade eden wrapper sınıftır. Null kontrolünü 'unutmayı' önler.

```
// Kötü kullanım
Musteri m = musterRepo.findByTckn(tckn);
String ad = m.getAd(); // m null ise NPE!

// İyi kullanım
Optional<Musteri> opt = musterRepo.findByTckn(tckn);
String ad = opt.map(Musteri::getAd).orElse("Bilinmeyen");

// Ya da:
Musteri m = opt.orElseThrow(() -> new MusteriBulunamadiException(tckn));
```

Anti-pattern: Field olarak Optional kullanma! Sadece return type olarak kullan. Method parametresi olarak da kullanma (overload yap).

3.4 Default ve Static Methods in Interface

SORU: Java 8'de interface'lere eklenen default ve static metodlar nedir?

Default metod: Interface içinde gövdesi olan metod. Implementing sınıflar override etmeden kullanabilir. Geriye uyumluluk (backward compatibility) için eklendi. Mesela List.sort() default metoddur, eski liste implementasyonları kırılmadan eklendi.

Static metod: Interface'in kendisine ait, instance olmadan çağrılır.

```
interface OdemeStrategy {
    void odemeYap(BigDecimal tutar);

    default void logla(BigDecimal tutar) {
        System.out.println("Ödeme: " + tutar);
    }

    static OdemeStrategy varsayilan() {
        return new HavaleStrategy();
    }
}
```

4. CONCURRENCY & MULTITHREADING

4.1 Thread vs Runnable

SORU: Thread mı Runnable mı kullanmalı? Neden?

İkisi de thread yaratır ama farklı yapıda:

- Thread'i extend edersen tek kalıtım hakkını harcarsın.
- Runnable bir interface'tir, implement edersen başka sınıfı da extend edebilirsin.
- Pratikte Runnable + ExecutorService kombinasyonu kullanılır.

```
// Direkt thread oluşturma - KÖTÜ
new Thread(() -> processOrder(order)).start();

// Modern yöntem - İYİ
ExecutorService executor = Executors.newFixedThreadPool(10);
executor.submit(() -> processOrder(order));

// Sonuç dönmesi gerekiyorsa Callable
Future<BigDecimal> future = executor.submit(() -> hesaplaBakiye(hesapId));
```

4.2 synchronized ve volatile

SORU: synchronized ve volatile arasındaki fark nedir?

synchronized: Bir bloğu/metodu aynı anda sadece 1 thread çalıştırabilir. Lock alır. Hem atomicity hem visibility sağlar.

volatile: Değişkenin değeri main memory'den okunur, thread cache'inden değil. Sadece visibility sağlar, atomicity sağlamaz. Yani counter++ volatile'la güvenli değildir!

```
// volatile - değişiklik anında diğer thread'ler görür
private volatile boolean durdurulduMu = false;

// synchronized - hem visibility hem atomicity
private int sayac = 0;
public synchronized void artir() { sayac++; }

// Atomic - en performanslı sayım için
private AtomicInteger sayac = new AtomicInteger(0);
sayac.incrementAndGet();
```

❑ Gerçek Hayat Örneği: Bankada bir ATM kuyruğunu düşün. Aynı anda 100 thread aynı hesaptan para çekiyorsa, bakiye alanı volatile bile olsa bakiye -= tutar race condition'a girer (oku-modifiye-yaz arasında başka thread araya girer). Burada synchronized veya pessimistic locking şart.

4.3 ExecutorService ve Thread Pool

SORU: Thread pool nedir? Neden kullanmalı?

Her istek için yeni thread oluşturmak çok pahalıdır (~1MB memory + OS context switch). Thread pool, önceden hazırlanmış thread'leri yeniden kullanır.

```
// Sabit boyut
ExecutorService fixed = Executors.newFixedThreadPool(10);
```

```
// İhtiyaca göre büyüyen
ExecutorService cached = Executors.newCachedThreadPool();

// Tek thread
ExecutorService single = Executors.newSingleThreadExecutor();

// Modern: parametrelili
ThreadPoolExecutor custom = new ThreadPoolExecutor(
    10, // core size
    50, // max size
    60L, TimeUnit.SECONDS, // keep alive
    new LinkedBlockingQueue<>(1000) // queue
);
```

4.4 CompletableFuture

SORU: CompletableFuture nedir? Future'dan farkı?

Future sadece get() ile blocking sonuç almanı sağlar. CompletableFuture ise non-blocking, zincirlenebilir async işlemler için.

```
// 3 farklı servisten paralel veri çek
CompletableFuture<Musteri> musterif = CompletableFuture.supplyAsync(
    () -> musteriservisi.getir(tckn));
CompletableFuture<List<Hesap>> hesaplarf = CompletableFuture.supplyAsync(
    () -> hesapservisi.getir(tckn));
CompletableFuture<KrediSkoru> skorf = CompletableFuture.supplyAsync(
    () -> kkbServisi.getir(tckn));

CompletableFuture.allOf(musterif, hesaplarf, skorf).join();

MusteriDetayResponse resp = new MusteriDetayResponse(
    musterif.get(), hesaplarf.get(), skorf.get());
```

□ Gerçek Hayat Örneği: Akbank müşteri profilini açtığında 5 farklı servisten veri çekiliyor: müşteri bilgisi, hesaplar, kredi kartları, kredi skoru, son işlemler. Senkron sıralı yaparsan toplam süre = her birinin toplamı (5 saniye). CompletableFuture ile paralel yaparsan en yavaş olanın süresi kadar (1 saniye). 5x hızlanma!

4.5 Deadlock

SORU: Deadlock nedir? Nasıl önlenir?

İki veya daha fazla thread'in birbirini sonsuza kadar beklediği durum. Thread A, B'nin elindeki kaynağı; Thread B, A'nın elindeki kaynağı bekler.

Önleme stratejileri:

- 1) Lock ordering: Tüm thread'ler kilitleri aynı sırada alsın.
- 2) tryLock() with timeout: Belirli süre bekle, alamazsan bırak.
- 3) Tek lock kullan (mümkünse).
- 4) Lock-free algoritmalar (AtomicReference, CompareAndSwap).

□ Gerçek Hayat Örneği: Para transferi yaparken iki hesabı kilitlemen gerekir. A'dan B'ye 100 TL gönderirken aynı anda biri B'den A'ya transfer yapıyorsa deadlock olur. Çözüm: hesap numarası küçük olan önce kilitlensin kuralı koy. Böylece tüm transferler aynı sırayla lock alır.

5. JVM, MEMORY & GARBAGE COLLECTION

5.1 JDK, JRE, JVM Farkları

SORU: JDK, JRE ve JVM arasındaki farklar nelerdir?

JVM (Java Virtual Machine): Bytecode'u çalıştıran sanal makine. Platform bağımlıdır (Windows JVM, Linux JVM farklı), bytecode platformdan bağımsızdır. 'Write once, run anywhere'.

JRE (Java Runtime Environment): JVM + standart kütüphaneler. Java çalıştırmak için yeter.

JDK (Java Development Kit): JRE + geliştirme araçları (javac, javadoc, jdb). Geliştirici olarak sen bunu kullanırsın.

Şema: JDK $\supset$ JRE $\supset$ JVM

5.2 JVM Memory Yapısı

SORU: JVM memory alanlarını anlat. Heap ve Stack arasındaki fark nedir?

Stack:

- Her thread için ayrı bir stack vardır.
- Metod çağrıları (stack frame) buraya konulur.
- Primitive değerler ve obje referansları burada tutulur.
- Metod bittiğinde frame otomatik temizlenir.
- Çok derin recursion $\rightarrow$ StackOverflowError.

Heap:

- Tüm thread'ler ortak kullanır.
- Tüm objeler (new ile yaratılanlar) burada.
- Garbage Collector burayı temizler.
- Heap dolarsa $\rightarrow$ OutOfMemoryError.

Metaspace (Java 8+, eskiden PermGen):

- Sınıf metadataları, statik field'lar.
- Native memory'de (heap dışında).
- -XX:MaxMetaspaceSize ile sınırlanabilir.

String Pool:

- Heap içinde özel bir alan.
- String literal'ler burada cache'lenir.
- "abc" == "abc" true döner.

```
void method() {
    int x = 5; // x  $\rightarrow$  stack
    Musteri m = new Musteri("Ali"); // m referansı  $\rightarrow$  stack
    // Musteri objesi  $\rightarrow$  heap
    // "Ali"  $\rightarrow$  string pool (heap içinde)
}
```

5.3 Garbage Collection

SORU: Garbage Collection nasıl çalışır? Generational GC nedir?

GC, artık kullanılmayan (unreachable) objeleri bellekten temizler. Programcı manuel free() yazmaz.

Generational hipotez: Çoğu obje çok kısa ömürlüdür. Buna göre heap bölünür:

- Young Generation: Yeni objeler. Üçe ayrılır: Eden, Survivor 0, Survivor 1.
- Minor GC: Young dolduğunda çalışır. Hızlıdır, eski objeleri Old'a taşır.
- Old Generation (Tenured): Uzun ömürlü objeler.
- Major/Full GC: Old dolduğunda. Yavaştır, uygulamayı durdurabilir (stop-the-world).

Modern GC algoritmaları:

- G1 (Garbage First): Java 9+ default. Heap'i region'lara böler, parça parça temizler.
- ZGC: <10 ms pause time, terabyte heap için.
- Shenandoah: Düşük pause time.

□ Gerçek Hayat Örneği: Bankacılıkta gece 02:00'de batch çalışan bir raporlama servisi var. Eğer Full GC olursa 5 saniye duraksıyor → toplam 8 saatlik batch'in ortasında müşteri timeout görür. Çözüm: G1 veya ZGC kullan, -Xms ve -Xmx aynı verilsin (heap büyüklüğünden gelen overhead'i önle), jstat ile izle.

5.4 Memory Leak

SORU: Java'da memory leak olur mu? Nasıl?

GC olsa bile olur. GC sadece unreachable objeleri toplar. Sen objeyi referans tutmaya devam edersen GC alamaz.

Tipik memory leak sebepleri:

- 1) Statik koleksiyonlar (örn. statik HashMap'e sürekli eklemek)
- 2) Unclosed resources (DB connection, file stream)
- 3) Inner class'ın outer class'ı tutması
- 4) ThreadLocal temizlenmemesi
- 5) Listener / Observer'ların unregister edilmemesi
- 6) HashMap'te equals/hashCode düzgün yapılmayınca aynı objelerin çoğalması

6. SPRING & SPRING BOOT

6.1 Spring vs Spring Boot

SORU: Spring ile Spring Boot arasındaki fark nedir?

Spring Framework: Tüm modüllerin (Core, MVC, Data, Security...) toplamı. Çok güçlü ama kurulumu zahmetli: XML/Java config, dependency'leri tek tek bulma, server kurma...

Spring Boot: Spring'in 'opinionated' (görüş bildiren) ve hızlı kurulan versiyonu:

- Auto-configuration: Sınıf yolundaki bağımlılıklara göre otomatik konfig.
- Starter dependencies: spring-boot-starter-web tek satırda web stack.
- Embedded server: Tomcat/Jetty/Undertow gömülü, jar olarak çalışır.
- Production-ready: Actuator ile health check, metrics, info endpoint'leri.
- application.properties / .yml ile config.

6.2 IoC ve Dependency Injection

SORU: IoC ve DI nedir? Aralarındaki ilişki?

IoC (Inversion of Control): Genel bir prensip. Nesnelerin yaşam döngüsünü ve bağımlılıklarını sen yönetmek yerine container'a bırakırsın. 'Sen beni arama, ben seni ararım' (Hollywood Principle).

DI (Dependency Injection): IoC'nin bir uygulama yöntemi. Bağımlılıklar dışarıdan (constructor, setter, field) verilir.

```
// IoC olmadan - Sıkı bağımlılık
class TransferServisi {
    private MailService mail = new MailService(); // KÖTÜ
}

// IoC ile - Constructor injection (EN İYİ)
@Service
class TransferServisi {
    private final BildirimServisi bildirim;

    public TransferServisi(BildirimServisi bildirim) {
        this.bildirim = bildirim;
    }
}
```

3 DI türü:

- 1) Constructor injection (tercih edilen): final yapılabilir, immutable, test kolay.
- 2) Setter injection: Opsiyonel bağımlılıklar için.
- 3) Field injection (@Autowired field üstünde): Önerilmez! Test zor, final yapılamaz, gizli bağımlılık.

6.3 Bean Scope'ları

SORU: Spring bean scope'ları nelerdir?

Scope	Açıklama	Kullanım
-------	----------	----------

singleton	Container başına 1 instance (default)	Stateless servisler
prototype	Her injection'da yeni instance	State tutan obje
request	Her HTTP request için yeni (web)	Request scoped data
session	Her HTTP session için yeni (web)	User session data
application	ServletContext başına 1	Tüm uygulama paylaşımı

6.4 Spring Stereotype Annotations

SORU: @Component, @Service, @Repository, @Controller farkları?

Aslında hepsi @Component'ten türer. Spring her birini bean olarak yaratır. Fark semantic (anlamsal) ve özel davranışlar:

- @Component: Genel bean. Hiçbir özel anlamı olmayan utility/helper.
- @Service: İş mantığı katmanı. Anlamsal işaret.
- @Repository: Veri erişim katmanı. Otomatik exception translation yapar (SQLException'ı DataAccessException'a çevirir).
- @Controller: MVC controller. @ResponseBody ile birlikte JSON döner.
- @RestController: @Controller + @ResponseBody.

6.5 @Transactional

SORU: @Transactional nasıl çalışır? Propagation ve isolation nedir?

@Transactional bir metoda eklenir, çağrı yapıldığında Spring otomatik olarak transaction başlatır, metod başarılıysa commit, exception olursa rollback yapar.

Arkada nasıl çalışır? Spring AOP ile proxy oluşturur. Sen metodu çağırdığında aslında proxy çağrılır, proxy önce transaction başlatır, sonra gerçek metodu çağırır.

```
@Service
public class TransferServisi {
    @Transactional
    public void transferYap(String fromId, String toId, BigDecimal tutar) {
        hesapRepo.paraCek(fromId, tutar); // 1
        if (kontrolBasarisiz()) {
            throw new RuntimeException(); // ROLLBACK!
        }
        hesapRepo.paraYatir(toId, tutar); // 2
    } // commit
}
```

Propagation seçenekleri:

- REQUIRED (default): Açık transaction varsa kullan, yoksa aç.
- REQUIRES_NEW: Her zaman yeni aç, mevcut varsa askıya al.
- NESTED: Mevcut içinde savepoint oluştur.
- SUPPORTS: Varsa kullan, yoksa transactionsız çalış.
- MANDATORY: Varsa kullan, yoksa exception.
- NEVER: Asla transaction olmasın.

Isolation seviyeleri:

- READ_UNCOMMITTED: Diğer transaction'ların commit edilmemiş datasını okuyabilir (dirty read).

- READ_COMMITTED: Sadece commit'lenmiş data. (PostgreSQL default)
- REPEATABLE_READ: Aynı satırı 2 kere okusan aynı değeri görürsün. (MySQL default)
- SERIALIZABLE: Tam izolasyon, en güvenli ama en yavaş.

□ Gerçek Hayat Örneği: Para transferinde READ_COMMITTED yeterli olur ama bakiye sorgulayıp ona göre işlem yapıyorsan (read-then-write) REPEATABLE_READ tercih et. Yoksa siz okurken başkası bakiyeyi değiştirebilir!

6.6 @Transactional Tuzakları

SORU: @Transactional ne zaman çalışmaz?

Spring proxy tabanlı çalışır, bazı tuzakları var:

1) Self-invocation (Aynı sınıf içinden çağrı):

```
@Service
public class Servis {
    public void disMetod() {
        icMetod(); // Proxy bypass! @Transactional çalışmaz!
    }

    @Transactional
    public void icMetod() { ... }
}
```

2) Private/final/static metodlarda çalışmaz (proxy override edemez).

3) Checked exception default'ta rollback yapmaz! @Transactional(rollbackFor = Exception.class) gerek.

4) try-catch içinde yutulursa rollback olmaz.

6.7 Spring Boot Auto-configuration

SORU: Spring Boot auto-configuration nasıl çalışır?

@SpringBootApplication aslında 3 annotation'ın birleşimidir: @Configuration + @EnableAutoConfiguration + @ComponentScan.

@EnableAutoConfiguration sayesinde, classpath'teki kütüphanelere göre Spring otomatik config eder:

- spring-boot-starter-web varsa → Tomcat + Spring MVC + Jackson kurulur.
- spring-boot-starter-data-jpa + H2 varsa → DataSource + EntityManager + JpaRepository.
- Mekanizma: META-INF/spring.factories (Boot 2) veya AutoConfiguration.imports (Boot 3) dosyalarında listelenen config sınıfları @ConditionalOnClass, @ConditionalOnMissingBean gibi koşullarla yüklenir.

```
// Sen Service yazmadıysan default oluştur:
@Configuration
@ConditionalOnClass(DataSource.class)
@ConditionalOnMissingBean(DataSource.class)
public class DataSourceAutoConfiguration { ... }
```

6.8 Spring Boot Profiles

SORU: Spring profile nedir? Nasıl kullanılır?

Farklı ortamlar (dev, test, prod) için farklı config kullanmamızı sağlar.

```
# application.yml (ortak)
spring:
  application:
    name: akbank-transfer-service

# application-dev.yml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/akbank_dev
  logging:
    level:
    root: DEBUG

# application-prod.yml
spring:
  datasource:
    url: jdbc:postgresql://prod-db:5432/akbank
  logging:
    level:
    root: WARN
```

Aktif profile: java -jar app.jar --spring.profiles.active=prod veya
SPRING_PROFILES_ACTIVE=prod env.

```
// Bean'i sadece belirli profile'da yarat
@Service
@Profile("dev")
public class MockOdemeServisi implements OdemeServisi { }

@Service
@Profile("prod")
public class GercekOdemeServisi implements OdemeServisi { }
```

6.9 Spring Boot Actuator

SORU: Spring Boot Actuator nedir?

Production'a hazır endpoint'ler sağlar: health check, metrics, info, env, thread dump, heap dump...

```
# pom.xml
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: always
```

Önemli endpoint'ler: /actuator/health, /actuator/metrics, /actuator/prometheus (Prometheus scraping için). Kubernetes liveness/readiness probe'larında /actuator/health/liveness ve /actuator/health/readiness.

6.10 Filter vs Interceptor

SORU: Filter ve Interceptor arasındaki fark nedir?

İkisi de request'leri yakalar ama farklı katmanlarda:

Özellik	Filter (Servlet)	Interceptor (Spring)
Katman	Servlet container	Spring MVC
Spring beans	Erişim yok	Erişim var
Çalışma zamanı	DispatcherServlet öncesi	Controller öncesi/sonrası
Kullanım	CORS, encoding, auth	Logging, user context, metrics

7. REST API VE WEB SERVICE TASARIMI

7.1 REST Nedir? REST Prensipleri

SORU: REST nedir? Temel prensipleri nelerdir?

REST (Representational State Transfer), Roy Fielding'in doktorasında tanımladığı bir mimari stildir. HTTP protokolünü kullanır.

Temel prensipler:

- 1) Client-Server: Sorumluluk ayrımı.
- 2) Stateless: Her request kendi içinde tam olmalı, sunucu state tutmamalı.
- 3) Cacheable: Cevaplar cache'lenebilir olmalı.
- 4) Uniform Interface: Kaynak (resource) tabanlı, URI ile tanımlı.
- 5) Layered System: Katmanlı yapı (load balancer, gateway, vb.).
- 6) Code on Demand (opsiyonel).

7.2 HTTP Metodları

SORU: HTTP metodlarını ve kullanımlarını anlat. Idempotent nedir?

Metod	Amaç	Idempotent?	Safe?
GET	Veri okuma	Evet	Evet
POST	Yeni kaynak yaratma	Hayır	Hayır
PUT	Tüm kaynağı güncelleme (replace)	Evet	Hayır
PATCH	Kısmi güncelleme	Genelde değil	Hayır
DELETE	Silme	Evet	Hayır

Idempotent: Aynı request'i N defa çağırırsan sunucudaki etkisi 1 defa çağırarak aynı olmalı.

❏ Gerçek Hayat Örneği: Para transferinde idempotency çok kritiktir! Müşteri 'Gönder' butonuna 3 kere bassın, ağ yavaş olsun → 3 transfer olmamalı. Çözüm: client her request'e benzersiz Idempotency-Key header'ı koyar, sunucu bu key'i Redis'te tutar, ikinci kez geldiyse cache'lenmiş cevabı döner.

7.3 HTTP Status Code'lar

SORU: HTTP status code'larını grupla. Hangi durumda hangisini dönersin?

2xx Başarı:

- 200 OK - genel başarı
- 201 Created - yeni kaynak oluştu (POST sonrası)
- 204 No Content - başarılı ama body yok (DELETE sonrası)

3xx Yönlendirme:

- 301 Moved Permanently
- 304 Not Modified - cache geçerli

4xx Client Hatası:

- 400 Bad Request - validation hatası
- 401 Unauthorized - kimlik doğrulanmamış
- 403 Forbidden - yetkin yok
- 404 Not Found - kaynak yok
- 409 Conflict - çakışma (örn. optimistic lock fail)
- 422 Unprocessable Entity - validasyon
- 429 Too Many Requests - rate limit

5xx Server Hatası:

- 500 Internal Server Error
- 502 Bad Gateway - upstream hatası
- 503 Service Unavailable
- 504 Gateway Timeout

7.4 API Versioning

SORU: REST API versioning nasıl yapılır?

- 1) URI versioning (en yaygın): /api/v1/musteriler, /api/v2/musteriler
- 2) Header versioning: Accept: application/vnd.akbank.v2+json
- 3) Query param: /api/musteriler?version=2 (önerilmez)
- 4) Content negotiation: Content-Type ile

□ Gerçek Hayat Örneği: Akbank mobil uygulamasının 5 milyon kullanıcısı var. Eski versiyonu hala kullananlar var (diyelim 50.000). API'yi kırarsan eski versiyon çöker. URI versioning ile /v1'i 6 ay daha tut, müşterileri /v2'ye migrate et, sonra /v1'i kapat.

7.5 PUT vs PATCH

SORU: PUT ve PATCH arasındaki fark nedir?

PUT: Tüm kaynağı değiştirir (replace). Eksik alan gönderirsen null olur.

PATCH: Sadece belirtilen alanları değiştirir (partial update).

```
// Mevcut müşteri: { ad: 'Ahmet', soyad: 'Yılmaz', telefon: '555' }

// PUT /musteriler/1 - Body: { ad: 'Veli' }
// Sonuç: { ad: 'Veli', soyad: null, telefon: null } // Tehlikeli!

// PATCH /musteriler/1 - Body: { ad: 'Veli' }
// Sonuç: { ad: 'Veli', soyad: 'Yılmaz', telefon: '555' } // İdeal
```

8. MICROSERVICES vs MONOLITH

8.1 Temel Fark

SORU: Monolitik ve microservices mimarileri arasındaki fark nedir?

Monolith: Tüm fonksiyonallite tek bir uygulamada (single deployable unit). Senin ARGUS gibi - tüm modüller (izin, eğitim, performans, vb.) aynı WAR/JAR içinde.

Microservices: Uygulama, iş kapasitelerine göre küçük, bağımsız servislere bölünür. Her servis kendi DB'si, kendi deployment'ı, kendi takımına sahip.

Konu	Monolith	Microservices
Deployment	Tek deployment	Bağımsız deployment
Database	Genelde tek shared DB	Her servis kendi DB'si
Teknoloji	Tek stack	Her servis farklı dil/DB seçebilir
Scale	Tüm uygulama ölçeklenir	Servis bazlı ölçekleme
Geliştirme hızı	İlk başta hızlı, sonra yavaşlar	İlk başta yavaş, sonra hızlı
Transaction	Tek DB → kolay (ACID)	Dağıtık → zor (SAGA, Eventual)
İletişim	In-process method call	HTTP/REST, gRPC, Messaging
Test	Kolay (tek uygulama)	Zor (entegrasyon karmaşık)
Operasyon	Basit (tek log)	Karmaşık (distributed tracing)
Maliyet	Düşük altyapı	Yüksek (K8s, monitoring, vb.)

8.2 Microservices Avantajları

- 1) Bağımsız deployment: Bir servisteki değişiklik diğerlerini etkilemez. Crash'lerse diğer servisler ayakta kalır (fault isolation).
- 2) Bağımsız ölçekleme: Sadece transfer servisi yoğunsa onu 10 instance'a çıkar, diğerleri 1'de kalsın. Para tasarrufu.
- 3) Teknoloji esnekliği: Bir servis Java, diğeri Python olabilir. ML servisi Python, transfer servisi Java.
- 4) Küçük takımlar: Conway's Law - 2 pizza takımı bir servis sahibi olur.
- 5) Daha hızlı CI/CD: Küçük servis = küçük test = hızlı pipeline.

8.3 Microservices Dezavantajları

- 1) Distributed system karmaşıklığı: Network latency, partial failure, distributed transaction.
- 2) Veri tutarlılığı zor: ACID yerine eventual consistency.
- 3) Debug/Monitoring zor: Bir transaction 10 servisten geçiyorsa loglara bakmak zor. Distributed tracing (Jaeger, Zipkin) şart.
- 4) Operasyonel yük: 50 servisi yönetmek - K8s, service discovery, config management.
- 5) Test karmaşıklığı: Contract testing, integration testing.
- 6) Cost: Daha fazla ağ trafiği, daha fazla hardware.

8.4 Ne Zaman Hangisi?

SORU: Hangi durumda monolith, hangi durumda microservices?

Monolith tercih edilebilir:

- Yeni başlayan startup (kullanıcı sayısı belirsiz)
- Küçük takım (<10 kişi)
- Domain henüz net değil
- Operasyonel maturity düşük (DevOps zayıf)

Microservices tercih edilebilir:

- Büyük organizasyon, çoklu takım
- Servisler farklı ölçekleme gerektiriyor
- Bağımsız deployment ihtiyacı (haftada birkaç release)
- Net domain sınırları belirlenmiş (DDD)
- Akbank gibi büyük bir banka için ideal.

□ Gerçek Hayat Örneği: Akbank'ta yaklaşımı düşün: mobil bankacılık ekibi ayrı, kredi kartı ekibi ayrı, krediler ekibi ayrı, ödeme ekibi ayrı. Her birinin kendi servisi, kendi DB'si, kendi release cycle'ı var. Mobil uygulama bir API Gateway üzerinden tümüne erişiyor. Bu sayede 'kart bonus hesaplama' güncellemesi 'havale' servisini hiç etkilemiyor.

8.5 Strangler Fig Pattern

SORU: Monolitten microservices'e nasıl geçilir?

Big Bang yapma! Hep birden değiştirirsen büyük risk alırsın. Strangler Fig Pattern ile aşamalı:

- 1) Monolitin önüne API Gateway koy.
- 2) Bir özelliği seç (en az bağımlı olan), yeni microservice olarak yaz.
- 3) Gateway'i, o endpoint'leri yeni servise yönlendirecek şekilde değiştir.
- 4) Monolitten o kodu kaldır.
- 5) Tekrarla. Yıllar sürebilir; sabır gerekir.

9. MICROSERVICES PATTERNS

9.1 SAGA Pattern (En Kritik Konu)

SORU: SAGA Pattern nedir? Neden ihtiyaç var?

Distributed transaction'lar için kullanılır. Microservices'te her servisin kendi DB'si olduğu için klasik ACID transaction yapamayız (2PC çok pahalı ve servis sayısı arttıkça performans öldürücü olur).

SAGA: Bir uzun transaction'ı, her biri lokal transaction olan adımlara böler. Bir adım başarısız olursa, daha önceki adımlar için compensating transaction (geri alma işlemi) çalıştırılır.

□ Gerçek Hayat Örneği: Akbank EFT Senaryosu:

- 1) Müşterinin hesabından 1000 TL düş (Hesap Servisi)
- 2) İşlem ücreti tahsil et 5 TL (Komisyon Servisi)
- 3) Karşı banka API'sine gönder (EFT Servisi)
- 4) İşlem kaydı tut (Audit Servisi)
- 5) SMS gönder (Bildirim Servisi)

Diyelim adım 3'te karşı banka reddetti. ACID olsaydı tek rollback. Ama burada her servis ayrı DB'de commit etti. Çözüm: SAGA

- Adım 1'in compensation'ı: Para iade et
- Adım 2'nin compensation'ı: Komisyonu iade et
- Adım 4'ün compensation'ı: İşlem kaydını iptal işle

9.2 SAGA: Orchestration vs Choreography

SORU: Orchestration ve Choreography arasındaki fark nedir?

1) Orchestration (Merkezi yönetici):

Bir 'orchestrator' (orquestra şefi) servisi tüm adımları sırayla çağırır, sonuçları takip eder. Hata olursa compensation'ları çağırır.

```
// EFT Orchestrator
@Service
public class EftOrchestrator {
    public void eftYap(EftRequest req) {
        try {
            hesapServisi.paraCek(req); // Adım 1
            komisyonServisi.tahsilEt(req); // Adım 2
            eftServisi.gonder(req); // Adım 3
            auditServisi.kaydet(req); // Adım 4
            bildirimServisi.smsGonder(req); // Adım 5
        } catch (EftFailedException e) {
            // Compensating transactions ters sırayla
            komisyonServisi.iadeEt(req);
            hesapServisi.paraIade(req);
        }
    }
}
```

Avantaj: Akış tek yerde, debug kolay.

Dezavantaj: Orchestrator single point of failure, sıkı bağlılık.

2) Choreography (Dağıtık):

Merkez yok. Her servis kendi adımını yapar ve event publish eder. Diğer servisler ilgili event'lere abone olur ve kendi işlerini yapar.

```
// Event flow
// Hesap Servisi:
publish("EFT_BAŞLATILDI");
@KafkaListener(topics = "EFT_BAŞLATILDI")
void handle(EftEvent e) {
    hesaptanCek(e);
    publish("PARA_CEKILDI");
}

// Komisyon Servisi:
@KafkaListener(topics = "PARA_CEKILDI")
void handle(...) {
    komisyonTahsilEt();
    publish("KOMISYON_TAHSIL_EDILDI");
}
// ... ve böyle devam eder
```

Avantaj: Gevşek bağıllık (loose coupling), her servis bağımsız.

Dezavantaj: Akışı izlemek zor (hangi servis ne yapıyor karmaşık), debug zor, döngüsel event'ler oluşabilir.

□ Gerçek Hayat Örneği: Akbank gibi büyük yerlerde genelde karma yaklaşım kullanılır. Kritik akışlar (para transferi) orchestration ile, asenkron yan etkiler (bildirim, loglama) choreography ile.

9.3 Outbox Pattern

SORU: Outbox Pattern nedir? Hangi sorunu çözer?

Problem: Servis hem DB'ye yazıyor hem mesaj kuyruğuna event publish ediyor. Bunlar farklı sistemler! Sırayla yaparsan ya 'DB yazdı ama event gitmedi' ya da 'event gitti ama DB rollback oldu' durumu doğar (dual write problem).

Çözüm: Event'i ayrı bir 'outbox' tablosuna aynı transaction içinde yaz. Ayrı bir worker (veya CDC - Change Data Capture) outbox'tan okuyup Kafka'ya yazsın. Böylece atomicity garantisi.

```
@Transactional
public void paraCek(String hesapId, BigDecimal tutar) {
    hesapRepo.bakiyeDus(hesapId, tutar); // Tablo 1

    OutboxEvent event = new OutboxEvent(
        UUID.randomUUID(),
        "PARA_CEKILDI",
        json(hesapId, tutar)
    );
    outboxRepo.save(event); // Tablo 2 - AYNI TRANSACTION!
} // Hepsi birden commit

// Ayrı bir scheduled job:
@Scheduled(fixedDelay = 1000)
public void publishOutboxEvents() {
    List<OutboxEvent> events = outboxRepo.findUnpublished();
    events.forEach(e -> {
        kafkaTemplate.send(e.getTopic(), e.getPayload());
        outboxRepo.markPublished(e.getId());
    });
}
```

9.4 Circuit Breaker Pattern

SORU: Circuit Breaker nedir? Nasıl çalışır?

Çağrılan servis düşük performansta veya çalışmıyor olabilir. Sürekli istek atarsan thread havuzunu doldurursun, kendi servisin de çöker → cascading failure.

Circuit Breaker elektrik sigortası gibi çalışır:

- CLOSED: Normal durum, istekler geçer. Hata sayısı izlenir.
- OPEN: Hata oranı eşiği aşınca açılır. İstekler hiç gitmez, anında fail döner (fail-fast). Belirli süre bekler.
- HALF-OPEN: Süre dolunca, 1-2 test isteği gönderir. Başarılıysa CLOSED'a, değilse tekrar OPEN'a döner.

```
// Resilience4j ile
@CircuitBreaker(name = "kkbServisi", fallbackMethod = "krediSkoruCacheTen")
public KrediSkoru getKrediSkoru(String tckn) {
    return kkbClient.skorGetir(tckn);
}

public KrediSkoru krediSkoruCacheTen(String tckn, Exception e) {
    return cache.getDefault(tckn, KrediSkoru.varsayilan());
}

# application.yml
resilience4j:
  circuitbreaker:
    instances:
      kkbServisi:
        failureRateThreshold: 50 # %50 fail olursa aç
        waitDurationInOpenState: 30s
        slidingWindowSize: 10
```

9.5 Bulkhead Pattern

SORU: Bulkhead Pattern nedir?

Gemi metaforu - geminin gövdesi bölmelere ayrılır, bir bölme su alırsa diğerleri ayakta kalır. Yazılımda: thread havuzlarını/connection pool'larını ayırmak.

□ Gerçek Hayat Örneği: Bir microservice'in hem KKB'ye hem kendi DB'sine çağrı yapan endpoint'leri var. KKB yavaşlarsa tüm thread'leri tüketir, normal DB sorguları da takılır. Bulkhead ile her bağımlılığa ayrı thread pool ver. KKB takılsın, DB endpoint'leri çalışmaya devam etsin.

9.6 CQRS

SORU: CQRS nedir?

Command Query Responsibility Segregation - okuma ve yazma modellerini ayırmak. Genelde Event Sourcing ile birlikte kullanılır.

□ Gerçek Hayat Örneği: Akbank müşteri hareket sorgusu örneği: müşteri 1 ay içinde sadece 5-10 işlem yapar, ama günde 100 kere son işlemlerini görmek ister. Write modeli normalize bir RDBMS'te tut (transaction integrity için), read modelini denormalize

Elasticsearch/Cassandra'da tut (hızlı sorgu için). Write tarafı event publish eder, read tarafı dinler ve güncellenir.

9.7 API Gateway

SORU: API Gateway nedir? Neden kullanılır?

Tüm client istekleri için tek giriş noktası. Spring Cloud Gateway, Kong, AWS API Gateway, Nginx gibi araçlarla yapılır.

Sorumlulukları:

- Routing (URL pattern'a göre doğru servise yönlendir)
- Authentication / Authorization (JWT validation)
- Rate limiting
- Request/Response transformation
- Aggregation (birkaç servisten veri çekip birleştirme)
- Logging, monitoring
- SSL termination
- Caching

☐ Gerçek Hayat Örneği: Akbank mobil uygulaması: tek bir endpoint'e (api.akbank.com.tr) bağlanır. Arkada Gateway, /musteriler'i müşteri servisine, /transferler'i transfer servisine, /kartlar'ı kart servisine yönlendirir. Mobil uygulama 50 servisin nerede olduğunu bilmek zorunda değil.

9.8 Service Discovery

SORU: Service Discovery nedir?

Microservice'lerin IP/port'ları sürekli değişir (özellikle K8s ortamında pod'lar yeniden başlar). Hard-coded IP yazamazsın. Servisler birbirini isim ile bulur.

Çözümler:

- Client-side discovery: Netflix Eureka, Consul. Client servis registry'den IP'leri çeker, kendi load balancing yapar.
- Server-side discovery: Kubernetes Service, AWS ELB. Yönlendirme infrastructure yapar.

10. INTER-SERVICE COMMUNICATION

10.1 Senkron vs Asenkron

SORU: Senkron ve asenkron iletişim arasındaki fark nedir? Ne zaman hangisi?

Senkron (REST, gRPC): Client istek atar, cevap bekler. Bağımlılık vardır.

Asenkron (Kafka, RabbitMQ, SQS): Mesaj kuyruğuna at, dön. Sonra biri okur, işler.

Durum	Tercih
Anında cevap gerekiyor (UI listening)	Senkron
İşlem fire-and-forget (bildirim)	Asenkron
Tight coupling istenmiyor	Asenkron
Sıralı işlem zinciri	Senkron veya Orchestration
Yüksek throughput, buffer ile	Asenkron
Birden çok consumer aynı event'i dinler	Asenkron (Pub-Sub)

10.2 Kafka vs RabbitMQ

SORU: Kafka ve RabbitMQ farkları nelerdir? Hangisi ne zaman?

Özellik	Kafka	RabbitMQ
Model	Log-based, persistent	Queue-based, transient
Throughput	Çok yüksek (milyonlar/sn)	Yüksek (10-100K/sn)
Mesaj saklama	Disk'te uzun süre tutar	Tüketildikten sonra silinir
Replay	Mümkün (offset)	Zor
Routing	Topic + partition	Exchange + binding (zengin)
Kullanım	Event streaming, log aggregation	Task queue, RPC

□ Gerçek Hayat Örneği: Akbank'ta ATM işlem event stream'i Kafka'ya gider (1 günlük log'u replay edebilmek için, audit, fraud detection ML modeli besleyebilmek için). Müşteriye gönderilecek SMS task'leri RabbitMQ'ya gider (priority queue, retry, dead letter).

10.3 Idempotency in Messaging

SORU: Mesajlaşmada idempotency nasıl sağlanır?

Kafka/RabbitMQ'da çoğu broker 'at-least-once' delivery garantisi verir. Yani mesaj 1 veya 1'den fazla kez gelebilir (network sorunu, consumer crash'i...). Aynı mesaj 2 kere işlenirse para 2 kere düşülmemeli!

Çözüm: Idempotency key

```
@KafkaListener(topics = "transfer-events")
@Transactional
public void handle(TransferEvent event) {
    // Daha önce işlendi mi?
    if (islemRepo.existsByEventId(event.getId())) {
        log.info("Duplicate, skipping");
        return;
    }

    // İşle
    hesapServisi.paraCek(event);

    // Kaydı tut
    islemRepo.save(new ProcessedEvent(event.getId()));
}
```

10.4 Dead Letter Queue

SORU: Dead Letter Queue nedir?

Bir mesaj N kere işlenemediği takdirde sürekli retry yapıp kuyruğu tıkamamak için DLQ'ya gönderilir. Manuel inceleme/işleme için.

□ Gerçek Hayat Örneği: EFT işleminde karşı banka API'sı bozuldu. Mesaj sürekli fail oluyor. 5 retry sonrası DLQ'ya atılır. Operasyon ekibi sabah DLQ'yu inceler, sorunu çözer, mesajları yeniden işler.

11. DATABASE & SQL

11.1 ACID Özellikleri

SORU: ACID nedir? Açar mısın?

Veritabanı transaction'larının 4 temel özelliği:

A - Atomicity (Atomiklik): Tümü ya da hiçbiri. Transaction içindeki tüm işlemler ya hep birlikte başarılı olur ya da hepsi geri alınır.

C - Consistency (Tutarlılık): Transaction sonunda DB tutarlı durumda olmalı. Constraint'ler (PK, FK, CHECK) ihlal edilmemeli.

I - Isolation (İzolasyon): Eşzamanlı transaction'lar birbirini bozmamalı. İzolasyon seviyesi ne kadar yüksekse o kadar güvenli ama yavaş.

D - Durability (Kalıcılık): Commit edilen bir transaction kalıcıdır. Sistem çökse bile kayıp olmaz (WAL - Write Ahead Log).

□ Gerçek Hayat Örneği: Para transferi: A hesabından B hesabına 1000 TL. 'A'dan düş, B'ye yatır' iki ayrı UPDATE. Atomicity: ikisi de olmalı ya da hiçbiri.
Consistency: Toplam para miktarı transfer sonrası aynı kalmalı.
Isolation: Başka biri aynı anda bakiye okumaya çalışırsa yarı işlemi görmemeli.
Durability: Commit'ten hemen sonra elektrik kesilse bile, açıldığında transfer yapılmış olmalı.

11.2 JOIN Türleri

SORU: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN farkları?

İki tablo (A, B) düşün:

- INNER JOIN: Sadece her iki tabloda da eşleşen kayıtlar.
- LEFT JOIN: A'nın tüm kayıtları + B'de eşleşen var ise onlar (yoksa NULL).
- RIGHT JOIN: Tersi - B'nin hepsi + A'da eşleşen.
- FULL OUTER JOIN: Her iki tablonun tüm kayıtları (eşleşme olmayanlar NULL).
- CROSS JOIN: Cartesian product. A'nın her satırı x B'nin her satırı.

```
-- Müşteri ve onların hesapları (hesabı olmayanları da göster)
SELECT m.ad, m.soyad, h.hesap_no, h.bakiye
FROM musteriler m
LEFT JOIN hesaplar h ON h.musteriler_id = m.id;

-- Sadece kredi kartı olan müşteriler
SELECT m.ad, k.kart_no
FROM musteriler m
INNER JOIN kredi_kartlari k ON k.musteriler_id = m.id;
```

11.3 Index Yapısı ve Türleri

SORU: Index nedir? Türleri nelerdir?

Index, kitapların index'i gibi - veriyi hızlı bulmak için ek veri yapısı. Olmasaydı veritabanı tüm tabloyu tarayacaktı (Full Table Scan).

Türleri:

- B-Tree Index: Default. Range query'ler için iyi (<, >, BETWEEN).
- Hash Index: Sadece eşitlik (=) için. Çok hızlı ama range yapamaz.
- Composite Index: Birden fazla kolon: (musteri_id, tarih). Leftmost-prefix kuralı: Index'i kullanmak için sorguda en soldaki kolon olmalı.
- Covering Index: SELECT'teki tüm kolonları içeren index. Tabloyu okumaya gerek yok.
- Partial Index: WHERE şartıyla oluşturulan (örn. sadece aktif=true olanlar).
- Full-text Index: Metin arama için.

□ Gerçek Hayat Örneği: ARGUS'taki TBCEDET gibi büyük bir tabloda WHERE musteri_id = ? sorgusu index olmadan 1 milyon satırı tek tek okur. Index ile $O(\log n)$ - direkt bulur. Ama dikkat: çok fazla index da yazma performansını düşürür (her INSERT'te tüm index'ler güncellenir). Dengeli olmalı.

11.4 N+1 Query Problem

SORU: N+1 Query Problem nedir? Nasıl çözülür?

ORM (JPA/Hibernate) ile en yaygın performans tuzaklarından biri. Bir liste çekersin, sonra her birinin ilişkili objesini çekersin → 1 + N query çalışır.

```
// PROBLEM
List<Musteri> musteriler = musteriRepo.findAll(); // 1 query
for (Musteri m : musteriler) {
    System.out.println(m.getHesaplar().size()); // Her biri için 1 query!
}
// 1000 müşteri → 1001 query!

// ÇÖZÜM 1: JOIN FETCH
@Query("SELECT m FROM Musteri m LEFT JOIN FETCH m.hesaplar")
List<Musteri> findAllWithHesaplar();

// ÇÖZÜM 2: EntityGraph
@EntityGraph(attributePaths = {"hesaplar"})
List<Musteri> findAll();

// ÇÖZÜM 3: Batch fetch
@BatchSize(size = 50)
@OneToMany ...
```

□ Gerçek Hayat Örneği: Akbank müşteri listesi ekranında 100 müşteri ve son işlemleri gösteriliyor. N+1 ile 101 sorgu, ekran 5 saniye yüklenir. JOIN FETCH ile 1 sorgu, 200ms'de açılır. Mülakatta 'Hibernate'te en sık karşılaştığın performans sorunu?' diye sorulursa bu, klasik cevaptır.

11.5 Optimistic vs Pessimistic Locking

SORU: Optimistic ve Pessimistic locking farkları nelerdir?

Pessimistic Locking: Önceden 'bu kayıt benim' diyerek kilitle. Diğerleri bekler. SQL'de SELECT ... FOR UPDATE.

Optimistic Locking: Kilitleme. Sadece bir version kolonu tut. Update ederken eski versiyonla geldiyse exception fırlat.

```
// Optimistic - JPA
@Entity
public class Hesap {
```

```
@Id Long id;
BigDecimal bakiye;

@Version
Long version; // Her update'te otomatik artar
}
// Update sırasında:
// UPDATE hesap SET bakiye=?, version=version+1
// WHERE id=? AND version=?
// Etkilenen satır 0 ise: OptimisticLockException

// Pessimistic - JPA
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT h FROM Hesap h WHERE h.id = :id")
Hesap findByIdForUpdate(@Param("id") Long id);
```

□ Gerçek Hayat Örneği: ATM'den para çekme - bakiye sorgula, kontrol et, düş. İki kişi aynı anda denese race condition. Pessimistic: hesabı kilitle, biri çekerken diğeri bekler (yüksek consistency). Optimistic: izin ver, biri başarısız olur (yüksek performans). Yüksek çakışma olasılığında pessimistic, düşükte optimistic.

11.6 Normalization vs Denormalization

SORU: Normalization nedir? 3NF ne demek?

Normalization: Tekrarı azaltmak ve veri bütünlüğünü artırmak için tabloları parçalama.

1NF: Her hücrede atomik değer (liste yok)

2NF: 1NF + her non-key kolon tüm primary key'e bağlı

3NF: 2NF + non-key kolonlar başka non-key kolonlara bağlı olmamalı (transitive bağımlılık yok)

Denormalization: Performans için bilerek tekrar ekleme. Read-heavy sistemlerde tercih edilir.

□ Gerçek Hayat Örneği: Akbank müşteri tablosunda şubeKodu varsa, şubeAdi'ni de tekrar etmek 3NF ihlali. Ama bir rapor servisi her sorguda JOIN yapmasın diye müşteri_rapor tablosu denormalize tutulabilir. OLTP normalize, OLAP denormalize.

12. CACHING & PERFORMANS

12.1 Caching Stratejileri

SORU: Cache stratejilerini açıkla.

1) Cache-Aside (Lazy Loading): En yaygın. Uygulama cache'i yönetir.

```
Musteri get(String tckn) {  
    Musteri m = cache.get(tckn);  
    if (m == null) {  
        m = db.findByTckn(tckn);  
        cache.put(tckn, m);  
    }  
    return m;  
}
```

2) Read-Through: Cache library DB'yi otomatik okur. Uygulama sadece cache'e konuşur.

3) Write-Through: Yazma hem cache'e hem DB'ye senkron. Tutarlı ama yavaş.

4) Write-Behind (Write-Back): Cache'e yaz, sonra async DB'ye. Hızlı ama DB'nin gerisinde kalabilir.

5) Refresh-Ahead: Süresi dolmadan önce yenile.

12.2 Cache Invalidation

SORU: 'Cache invalidation' neden zor?

Phil Karlton'ın ünlü sözü: 'Bilgisayar biliminde sadece 2 zor şey vardır: cache invalidation ve isimlendirme.'

Stratejiler:

- TTL (Time To Live): Belli süre sonra otomatik sil. Basit ama eski veri olabilir.
- Event-based: Veri değişince ilgili cache key'i sil. Karmaşık ama güncel.
- Versioning: Cache key'e versiyon ekle, versiyonu artırınca eskiler ölür.

❏ Gerçek Hayat Örneği: Akbank müşteri bilgisi cache'lendi. Müşteri telefonunu güncelledi. Eğer 1 saatlik TTL varsa 1 saat eski veriyi görürsün. Event-based ile MUSTERI_GUNCELLENDI event'i geldiğinde cache.evict("musteri:" + tckn) ile sil. Bir sonraki get'te DB'den okuyacak.

12.3 Spring Cache + Redis

SORU: Spring'de cache nasıl entegre edilir?

```
// 1) Bağımlılık ekle  
// spring-boot-starter-data-redis, spring-boot-starter-cache  
  
// 2) Etkinleştir  
@SpringBootApplication  
@EnableCaching  
public class Application { ... }  
  
// 3) Kullan  
@Service  
public class MusteriServisi {  
    @Cacheable(value = "musteriler", key = "#tckn")  
    public Musteri get(String tckn) {
```

```
return musterRepo.findByTckn(tckn);
}

@CacheEvict(value = "musteriler", key = "#m.tckn")
public void update(Musteri m) {
    musterRepo.save(m);
}

@CachePut(value = "musteriler", key = "#m.tckn")
public Musteri create(Musteri m) {
    return musterRepo.save(m);
}
}
```

12.4 Cache Stampede ve Çözümleri

SORU: Cache stampede nedir? Nasıl önlenir?

Popüler bir cache key'in expire olduğu anda 1000 thread aynı anda cache miss yapıp DB'ye saldırması durumu (Thundering Herd). DB çöker.

Çözümler:

- Lock/Mutex: İlk thread DB'den çekerken diğerleri bekler.
- Refresh-Ahead: Expire öncesi proaktif yenile.
- Jittered TTL: Aynı zamanda expire olmasınlar diye TTL'e random ekle.

13. SECURITY

13.1 Authentication vs Authorization

SORU: Authentication ve Authorization farkı?

Authentication (Kimlik doğrulama): Sen kimsin? Kullanıcı adı/şifre, biometrik, OTP, sertifika ile sorulur.

Authorization (Yetkilendirme): Bu işlemi yapma yetkin var mı? Rol, izin, scope ile kontrol edilir.

Gerçek Hayat Örneği: Akbank online şubesine giriş yaptın → Authentication. Ama 100K TL üzeri transfer yapmak için ek onay (3D Secure) istedi → Authorization (yetki kontrolü).

13.2 JWT (JSON Web Token)

SORU: JWT nedir? Nasıl çalışır?

Stateless authentication için kullanılan standart token formatı. Header.Payload.Signature şeklinde 3 parçadan oluşan base64-encoded string.

Header: Algoritma (HS256, RS256) ve tip

Payload: Claim'ler (kullanıcı id, rol, exp, iat)

Signature: Header+Payload'ın secret ile imzalanmış hali

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJlc2VyMSIsImV4cCI6MTcwMH0 . abc123sig
```

Akış:

- 1) User login → server JWT üretir, döner.
- 2) Client her request'te Authorization: Bearer <token> header'ı koyar.
- 3) Server signature'ı verify eder. Geçerliyse claim'lere göre işlem yapar.
- 4) Server JWT'yi DB'de tutmaz (stateless).

Avantaj: Stateless, mikroservislerde paylaşılabilir.

Dezavantaj: Logout/revoke zor (token süresi dolmadan iptal edemezsin). Çözüm: Kısa exp + refresh token + blacklist.

13.3 OAuth 2.0

SORU: OAuth 2.0 nedir? Authorization flow'ları nelerdir?

OAuth 2.0 bir authorization framework'üdür. Üçüncü taraf uygulamalara sınırlı erişim vermeyi sağlar.

Roller:

- Resource Owner: Kullanıcı
- Client: Erişim isteyen uygulama
- Authorization Server: Token veren
- Resource Server: Korumalı kaynak (API)

Grant türleri:

- Authorization Code: Server-side web uygulamaları için (en güvenli).
- Implicit: Eskiden SPA için, artık önerilmez.
- Client Credentials: Servis-to-servis (kullanıcı yok).

- Refresh Token: Yeni access token almak için.
- PKCE: Mobile/SPA için güvenli versiyon.

13.4 Spring Security Temelleri

SORU: Spring Security nasıl çalışır?

Spring Security, request'lerin önüne filter chain koyar. Her filtre bir şeyi kontrol eder (authentication, CSRF, CORS, vb.).

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    SecurityFilterChain chain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
            .sessionManagement(s -> s.sessionCreationPolicy(
                SessionCreationPolicy.STATELESS))
            .build();
    }
}
```

13.5 Yaygın Saldırıları

SORU: SQL Injection, XSS, CSRF nedir? Nasıl önlenir?

SQL Injection: Kullanıcı girdisinin SQL sorgusuna doğrudan eklenmesi.

Önleme: Prepared Statement / Parameterized Query kullan. ORM zaten yapıyor.

```
// KÖTÜ - SQL Injection açığı
String sql = "SELECT * FROM musteriler WHERE tcno='" + input + "'";

// İYİ - Prepared statement
PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM musteriler WHERE tcno=?");
ps.setString(1, input);
```

XSS (Cross-Site Scripting): User input'a script enjekte edilmesi.

Önleme: Output encoding (HTML escape), Content Security Policy, HttpOnly cookie.

CSRF (Cross-Site Request Forgery): Kullanıcının bilgisi olmadan adına işlem yaptırma.

Önleme: CSRF token (Spring Security default), SameSite cookie.

14. DESIGN PATTERNS

14.1 Singleton

SORU: Singleton pattern nedir? Thread-safe nasıl yapılır?

Tek instance garantisi. Spring bean'leri default singleton'dır.

```
// Klasik thread-safe singleton (Double-Checked Locking)
public class Config {
    private static volatile Config instance;
    private Config() {}

    public static Config getInstance() {
        if (instance == null) {
            synchronized (Config.class) {
                if (instance == null) {
                    instance = new Config();
                }
            }
        }
        return instance;
    }
}

// En iyi yöntem: Enum singleton
public enum Config {
    INSTANCE;
    // metodlar...
}
```

14.2 Factory Pattern

SORU: Factory pattern nedir?

Objeye yaratma mantığını gizler. Client new kullanmaz, factory'ye 'şunu istiyorum' der.

```
public interface OdemeYontemi {
    void odemeYap(BigDecimal tutar);
}

public class OdemeFactory {
    public static OdemeYontemi create(OdemeTipi tip) {
        switch (tip) {
            case KREDI_KARTI: return new KrediKartiOdeme();
            case HAVALE: return new HavaleOdeme();
            case CRYPTO: return new CryptoOdeme();
            default: throw new IllegalArgumentException();
        }
    }
}
```

14.3 Builder Pattern

SORU: Builder pattern ne işe yarar?

Çok parametrelili constructor yerine fluent API ile obje oluşturma. Lombok'ta @Builder.

```
Musteri m = Musteri.builder()
    .ad("Ahmet")
    .soyad("Yılmaz")
    .tckn("12345678901")
    .telefon("05551234567")
    .email("a@b.com")
    .build();
```

14.4 Strategy Pattern

SORU: Strategy pattern nedir?

Algoritma ailesi tanımla, her birini ayrı sınıfa koy, runtime'da değiştirilebilir yap. Open/Closed prensibinin yardımcısı.

```
public interface FaizStrategy {
    BigDecimal hesapla(BigDecimal anapara, int gun);
}

public class SabitFaizStrategy implements FaizStrategy { ... }
public class DegiskenFaizStrategy implements FaizStrategy { ... }
public class IslamiBankFaizStrategy implements FaizStrategy { ... }

public class KrediHesaplayici {
    private final FaizStrategy strategy;
    // Constructor injection ile farklı stratejiler
    public BigDecimal toplamFaiz(BigDecimal anapara, int gun) {
        return strategy.hesapla(anapara, gun);
    }
}
```

14.5 Observer Pattern

SORU: Observer pattern?

Bir nesnedeki değişikliği takipçilere (observer) bildir. Spring Events buna örnek.

```
// Spring Events
// Publisher
applicationEventPublisher.publishEvent(new TransferTamamlandiEvent(transfer));

// Listener
@EventListener
public void smsGonder(TransferTamamlandiEvent event) {
    smsService.gonder(event.getMusteriTel(), "Transfer onaylandı");
}

@EventListener
public void auditLogYaz(TransferTamamlandiEvent event) {
    auditRepo.save(...);
}
```


15. BANKACILIK SENARYOLARI

15.1 Para Transferi - Klasik Senaryo

SORU: A hesabından B hesabına para transferini nasıl güvenli yaparsın?

Bu mülakatın klasiği. Birkaç katmanlı düşün:

1) Single DB - Monolitte: @Transactional + pessimistic lock ile atomicity sağlanır.

```
@Transactional
public void transfer(Long fromId, Long toId, BigDecimal tutar) {
    // Deadlock önlemek için ID'ye göre sırala
    Long firstId = Math.min(fromId, toId);
    Long secondId = Math.max(fromId, toId);

    Hesap h1 = hesapRepo.findByIdForUpdate(firstId);
    Hesap h2 = hesapRepo.findByIdForUpdate(secondId);

    Hesap from = h1.getId().equals(fromId) ? h1 : h2;
    Hesap to = h1.getId().equals(toId) ? h1 : h2;

    if (from.getBakiye().compareTo(tutar) < 0) {
        throw new YetersizBakiyeException();
    }

    from.setBakiye(from.getBakiye().subtract(tutar));
    to.setBakiye(to.getBakiye().add(tutar));

    // Audit log
    transferLogRepo.save(new TransferLog(fromId, toId, tutar));
}
```

2) Microservices - Farklı servislerse: SAGA Pattern + Outbox Pattern kullan. Para çekme, yatırma, audit ayrı servislerde.

3) Veri tipi: Asla double/float kullanma! BigDecimal kullan, floating-point hatası 1 kuruşu kaybettirir.

4) Idempotency: Müşteri 'Gönder'e 3 kere bassın 1 transfer olsun. Idempotency key kullan.

5) Audit: Her transfer log'a yazılmalı. Compliance ve müşteri itirazları için.

15.2 Aynı Anda Yüklenen Sistem (High Concurrency)

SORU: ATM'de 10.000 kişi aynı anda 'bakiye sorgula' yapıyor. Nasıl handle ederim?

Strateji:

- 1) Caching: Bakiye Redis'te tutulsun. 5-10 saniye TTL.
- 2) Read replica: DB'nin read replica'larını kullan, write master ayrı.
- 3) Connection pooling: HikariCP doğru yapılandırılmış olmalı.
- 4) Async: İşlem sonrası push notification async olsun.
- 5) Rate limiting: Bir IP'den dakikada N istek.
- 6) Auto-scaling: K8s HPA ile yük artınca otomatik pod eklensin.

15.3 Eventual Consistency Tolere Edilir mi?

SORU: Bankada eventual consistency kabul edilir mi?

Duruma göre deęiřir:

- Strong consistency gerekir: Para transferi, ödeme, hesap bakiyesi (anlık).
- Eventual consistency yeter: Müřteri profil bilgisi, ürün katalogu, hareket geçmiři (birkaç saniye gecikme tolere edilebilir), bildirimler.

□ Gerçek Hayat Örneęi: EFT yaptıktan 5 saniye sonra karşı tarafa para geçer (eventual). Ama kendi bakiyeden anında düşer (strong). UX olarak müşteriye 'işlem alındı' der, arka planda işler.

15.4 Distributed Tracing

SORU: Bir işlem 10 servisten geçiyor, hata oldu. Nasıl bulursun?

Distributed Tracing şart. Her request'e traceId (unique) verilir, tüm servisler bu ID'yi log'a yazar. Çocuk request'lere de propagate edilir.

Araçlar: Spring Cloud Sleuth (Boot 2.x) / Micrometer Tracing (Boot 3.x), Zipkin, Jaeger, OpenTelemetry.

```
// application.yml
management:
  tracing:
    sampling:
      probability: 1.0
    zipkin:
      tracing:
        endpoint: http://zipkin:9411/api/v2/spans

// Log'larda otomatik:
// [transfer-svc, traceId=abc123, spanId=def456] EFT başlatıldı
// [hesap-svc, traceId=abc123, spanId=ghi789] Para çekildi
// [bildirim-svc, traceId=abc123, spanId=jkl012] SMS gönderildi
```

15.5 Log Strategy

SORU: Logging stratejin ne? Production'da nasıl monitor edersin?

- 1) Yapılandırılmış log: JSON format. Logstash/Splunk/ELK ile parse edilebilir.
- 2) Log levels doğru:
 - ERROR: Müdahale gerektiren sorun
 - WARN: Beklenmedik ama akış devam ediyor
 - INFO: Önemli iş akışları (transfer başladı, tamamlandı)
 - DEBUG: Geliştirme için detay
- 3) PII (Personal Identifiable Information) log'a yazma! TCKN, kart no, şifre asla.
- 4) traceId her log'da olsun.
- 5) Centralized logging: ELK Stack (Elasticsearch + Logstash + Kibana).
- 6) Metrics: Prometheus + Grafana ile dashboard.
- 7) Alerting: Error rate, latency p99, throughput eşiklerinde alarm.

15.6 Soft Skills / Davranışsal

Akbank teknik mülakatlarında soft skill soruları da bekleyebilirsiniz:

- 'Zorlu bir problem nasıl çözdün?' → STAR yöntemi (Situation, Task, Action, Result). Senin Tostyalı yarım gün izin bug'ı veya PDF rendering issue'u harika örnekler.
- 'Takım arkadaşınla anlaşmazlık?' → Sakin yaklaşım, dürüst iletişim, ortak hedef vurgusu.
- 'Code review'da disagreement?' → Veriye dayalı tartışma, ego'yu bırakma, test case ile ispat.
- 'En çok gurur duyduğun proje?' → ARGUS'taki yeni bulk upload screen'i, veya VBECDDET view extension'ı; etkisini ölç (zaman tasarrufu, kullanıcı sayısı...).
- 'Neden Akbank?' → Önceden hazırlıklı git: Akbank LAB (Akbank'ın inovasyon merkezi), mobil uygulama, fintech yaklaşımı, ölçek, modern stack.

16. SON TÜYOLAR VE STRATEJİ

Mülakat Sırasında Altın Kurallar

- 1) Düşün, sonra konuş. Soruyu duyup hemen konuşma. 5 saniye düşün, ne diyeceğini planla. Sessizlik korkutucu değil, dengesizlik korkutucudur.
- 2) 'Bilmiyorum' demek serbest. Ama 'Bilmiyorum ama mantığı şu olmalı...' diye devam et. Mülakatçı seninle muhakeme yapmaya bayılır. Bilmediğini bilen biri, bilgisizliğini saklayandan iyi.
- 3) Trade-off konuş. Hiçbir teknik silver bullet değil. 'X iyidir ama Y dezavantajı var, şu durumda Z tercih edilebilir.' Senior'lık böyle ölçülür.
- 4) Gerçek tecrübeyi araya kat. 'ARGUS'ta benzer bir sorunla karşılaştığımda...', 'Tosyalı izin bug'unda compensating logic yazmıştık...' gibi gerçek hayat referansları.
- 5) Soru sor. 'Akbank'ta hangi cache çözümünü kullanıyorsunuz?', 'Microservices stack'iniz nedir?', 'Deployment yapımız nasıl?'. İlgi gösterdiğini gösterir, hem de mülakatçıyı konuşturursun (rahatlama).
- 6) Beden dili. Online mülakatsa kameraya bak, dik dur, gül. Stresliyen nefes al.

Beklenebilecek Coding Soruları

1 saatlik mülakatta uzun kod istemezler ama küçük şeyler bekleyebilirsiniz:

- String'i ters çevir
- Palindrom kontrolü
- Stream API ile listeyi grupta
- FizzBuzz
- Bir cümledeki kelimeleri tekrar say (Map)
- Singleton implement et
- REST endpoint yaz (mental olarak)
- SQL: 2. en yüksek maaşı bul

```
-- 2. en yüksek maaş
SELECT MAX(maas) FROM personel
WHERE maas < (SELECT MAX(maas) FROM personel);

-- Veya pencere fonksiyonu ile
SELECT DISTINCT maas FROM (
  SELECT maas, DENSE_RANK() OVER (ORDER BY maas DESC) AS rk
  FROM personel
) t WHERE rk = 2;
```

Hızlı Tekrar Listesi (Mülakattan 1 Saat Önce)

OOP: 4 prensip (Encapsulation, Inheritance, Polymorphism, Abstraction).
SOLID: SRP, OCP, LSP, ISP, DIP.
String/SB: Immutable, thread-safe vs hızlı.
HashMap: Bucket array + LinkedList → Java 8'de Tree. Load factor 0.75.
Stream: Intermediate lazy, terminal eager.
Volatile vs Synchronized: Visibility only vs Visibility+Atomicity.

GC: Young (Eden+S0+S1), Old. Minor/Major GC. G1.
Spring: IoC, DI types (constructor > setter > field).
@Transactional: Self-invocation çalışmaz, checked exception rollback yapmaz.
Mono vs Micro: Deployment, DB, scale, complexity.
SAGA: Compensating transactions, Orchestration vs Choreography.
Outbox: Dual write problem çözer.
Circuit Breaker: CLOSED → OPEN → HALF-OPEN.
ACID: Atomicity, Consistency, Isolation, Durability.
Isolation: Read Uncommitted < Committed < Repeatable < Serializable.
Optimistic vs Pessimistic: Version field vs SELECT FOR UPDATE.
N+1: JOIN FETCH, EntityGraph.
JWT: Header.Payload.Signature. Stateless.
HTTP idempotent: GET, PUT, DELETE; POST/PATCH değil.

Son Söz

Batur, sen 4 yıldır gerçek üretim sistemleri yazıyorsun: ARGUS'taki transaction yönetimi, REST servisler, bulk upload optimizasyonları, performans debug'ları... Bunlar zaten mülakatta soracakları konuların gerçek hayat karşılığı. Bilgini güveniyle birleştir, düşünerek konuş, dürüst ol.

Bilmediğin bir şey çıkarsa endişelenme. Hiç kimse her şeyi bilmek zorunda değil. Önemli olan nasıl düşündüğün ve nasıl öğrendiğin. Bu PDF'i bir kez baştan sona, bir kez de kendi cümlelerinle anlat. Mülakata gittiğinde rahat olacaksın.

Akbank seni alacak. ☐

Bol şans, kolay gelsin! ☐